Rational Matrix Policy Optimization for Rational Feed-Forward Language Models

Anonymous authors
Paper under double-blind review

Abstract

1 Introduction

2 Background and Related Work

3 Rational Matrix Policy Optimization

Let $\mathcal{L}(\theta)$ be the language-model training objective. For layer l, the RLB block projects the residual-stream input $x_l \in \mathbb{R}^d$ into G groups of width m,

$$z_l = A_l x_l, \qquad z_{l,g} = \mathrm{group}_g(z_l), \tag{1}$$

normalizes each group by its RMS radius,

$$r_{l,g} = \sqrt{m^{-1}\|z_{l,g}\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u_{l,g} = z_{l,g}/r_{l,g}, \tag{2}$$

and applies a learned rational map in normalized coordinates,

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \qquad y_l = B_l \,\mathrm{concat}_{g=1}^{G}\, h_{l,g}. \tag{3}$$

The input matrix A_l selects rational domains, the coefficients of $R_{l,g}$ shape the group response, and the output matrix B_l recombines rational features. MatrixPolicy uses this separation by partitioning parameters as

$$\theta = \theta_{\mathrm{base}} \cup \theta_{\mathrm{coeff}} \cup \{A_l, B_l\}_{l=1}^{L}. \tag{4}$$

The base parameters use the same optimizer as the non-rational baseline. Rational coefficients use a coefficient rule. MatrixPolicy is applied only to A_l and B_l.

In the homogeneous case $\epsilon_{\mathrm{rms}} = 0$, and for nonzero group radii, each group has a positive scale gauge:

$$A_{l,g} \mapsto a_g A_{l,g}, \qquad B_{l,g} \mapsto a_g^{-1} B_{l,g}, \qquad a_g > 0. \tag{5}$$

The represented function is unchanged because $z_{l,g}$ and $r_{l,g}$ scale together, $u_{l,g}$ is unchanged, and the inverse scale in B_l cancels the scale in $h_{l,g}$. With a nonzero RMS floor this symmetry is only approximate, so we use the gauge as an optimizer-design principle rather than as a formal invariance claim. This matters because optimizer state is stored in coordinates, not in the quotient space of represented functions.

At step t, each matrix role $r \in \{\mathrm{in}, \mathrm{out}\}$ receives a role-depth factor $\rho_r(l)$ and an on-policy statistic factor $s_{\mathrm{stat}}(l, r, t)$. The matrix update is

$$\begin{aligned} M_{l,r}^{t+1} = M_{l,r}^{t} &+ \Delta_{\mathrm{adapt}}(M_{l,r}; \eta_t a_{\mathrm{mat}}(l,r,t)[1-\mu(l,r,t)]) \\ &+ \Delta_{\mathrm{mat}}(M_{l,r}; \eta_t a_{\mathrm{matnorm}} \mu(l,r,t))\,, \end{aligned} \tag{6}$$

where $M_{l,\mathrm{in}} = A_l$, $M_{l,\mathrm{out}} = B_l$, η_t is the shared base learning rate, Δ_{adapt} is an Adam-style adaptive step, and Δ_{mat} is a short early matrix-normalized step. The mixture coefficient

$\mu(l, r, t)$ is active only early in training. After the matrix update, MatrixPolicy computes a target log-ratio for each group and applies the bounded rebalance

$$A_{l,g} \leftarrow e^{\ell_g} A_{l,g}, \qquad B_{l,g} \leftarrow e^{-\ell_g} B_{l,g}. \tag{7}$$

The update changes the representative used by the optimizer while preserving, or with $\epsilon_{\text{rms}} > 0$ approximately preserving, the represented block function. Full hyperparameter definitions are in Appendix A.

Algorithm 1: Rational Matrix Policy step. Given gradients $\nabla\mathcal{L}(\theta^t)$ and base learning rate η_t:

1. Update exponential moving averages of rational-group pressures.
2. Compute role-depth and group-stat multipliers for each RLB matrix.
3. Update non-rational parameters with the base optimizer and update rational coefficients with the coefficient rule.
4. Update each A_l and B_l with the MatrixPolicy mixture step.
5. Apply the bounded positive-gauge rebalance to each rational group.

4 Experiments

5 Discussion

6 Conclusion

A MatrixPolicy Details

Role-depth factors. For layers indexed by $l = 1, \ldots, L$ with $L > 1$, define normalized layer depth $d_l = (l-1)/(L-1)$ and set

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(d_l - 0.5), 0.55, 1.40), \tag{8}$$
$$\rho_{\text{out}}(l) = \text{clip}(1 + 1.00(d_l - 0.5), 0.55, 1.40), \tag{9}$$
$$a_{\text{role}}(l, r) = \max(0.10, 1 + 1.20(\rho_r(l) - 1)). \tag{10}$$

Let $\xi_t = t/T$ be training progress and let $\sigma_{a,b}(\xi)$ denote the smoothstep gate from 0 to 1 over $[a, b]$. The statistic multiplier $a_{\text{stat}}(l, r, t) > 0$ is the layer/role aggregate of the group preconditioners defined below, normalized to one when group statistics are not active. The local adaptive scale is

$$a_{\text{mat}}(l, r, t) = \text{clip}(3.0\, a_{\text{role}}(l, r) a_{\text{stat}}(l, r, t), 0.40, 4.0). \tag{11}$$

The matrix-normalized branch uses multiplier $a_{\text{matnorm}} > 0$ and mixture weight $\mu(l, r, t) \in [0, 1]$, a smoothstep-gated function of ξ_t, layer depth, matrix role, and $a_{\text{stat}}(l, r, t)$.

On-policy group statistics. All ratios and logarithms below are evaluated on strictly positive stabilized RMS, EMA, and gain quantities; the displayed formulas omit the corresponding epsilon floors. For group g, MatrixPolicy tracks relative gradient pressures

$$p_{\text{in},g} = \frac{\text{rms}(\nabla A_{l,g})}{\text{rms}(A_{l,g})}, \qquad p_{\text{out},g} = \frac{\text{rms}(\nabla B_{l,g})}{\text{rms}(B_{l,g})}, \qquad p_{\text{rat},g} = \text{rms}(\nabla\theta_{\text{coeff},g}). \tag{12}$$

With EMAs $q_{\text{in},g}$, $q_{\text{out},g}$, and $q_{\text{rat},g}$, we define

$$\pi_g = \log q_{\text{in},g} - \log q_{\text{out},g}, \qquad \alpha_g = \log q_{\text{rat},g} - \frac{1}{2}(\log q_{\text{in},g} + \log q_{\text{out},g}). \tag{13}$$

Group-stat preconditioning. The group preconditioner is centered and clipped:

$$c_g = c_{\text{gain},g} c_{\text{pressure},g} c_{\text{activity},g}, \qquad c_g \leftarrow c_g / \operatorname{geomean}(c), \qquad c_g \leftarrow \operatorname{clip}(c_g, 0.75, 1.35). \tag{14}$$

Here $k_g > 0$ is the role-specific group gain used for matrix preconditioning: the derivative-RMS gain for A_l and the output-RMS gain for B_l. The factors are

$$c_{\text{gain},g} = \left(\frac{\operatorname{geomean}(k)}{k_g} \right)^{0.20}, \qquad c_{\text{pressure},g} = \exp(0.10\, d_{\text{pressure},g}), \tag{15}$$

where $d_{\text{pressure},g} = -\pi_g$ for A_l and $+\pi_g$ for B_l. The activity damping term is

$$c_{\text{activity},g} = \exp[-0.20\ \operatorname{ReLU}((\alpha_g - 0.05)/0.45)]. \tag{16}$$

Gauge rebalance. Let

$$\gamma_g = \log \operatorname{rms}(A_{l,g}) - \log \operatorname{rms}(B_{l,g}). \tag{17}$$

The target log-ratio is

$$\tau_g = \frac{1}{2}(\log d_g^{\text{gain}} - \log o_g^{\text{gain}}) + \beta_{\text{pressure}}(\log q_{\text{in},g} - \log q_{\text{out},g}). \tag{18}$$

Here $d_g^{\text{gain}} > 0$ and $o_g^{\text{gain}} > 0$ are the derivative and output group-gain estimates, and β_{pressure} is the pressure balance coefficient. Let $s(t,l) \in [0,1]$ be the rebalance schedule, $a_g^{\text{activity}} \in (0,1]$ the activity damping multiplier, and $\ell_{\max} > 0$ the maximum absolute log-scale move. The exact target step is $\ell_g^\star = \frac{1}{2}(\tau_g - \gamma_g)$, while the applied bounded move is

$$\ell_g = \operatorname{clip}(s(t,l) a_g^{\text{activity}} \ell_g^\star, -\ell_{\max}, \ell_{\max}). \tag{19}$$

For true RMS values, the clipped update maps γ_g to $\gamma_g + 2\ell_g$ and is a bounded move toward τ_g rather than exact canonicalization unless clipping and schedule damping are inactive. With stabilized RMS values this identity is approximate; the implementation uses the bounded move as a controlled representative update for the RLB matrices.